

HAPPY NUMBER

```
class Solution:
```

```
    def isHappy(self, n):
```

```
        seen = set()
```

```
        while n != 1:
```

```
            if n in seen:
```

```
                return False
```

```
            seen.add(n)
```

```
            total = 0
```

```
            while n > 0:
```

```
                digit = n % 10
```

```
                total += digit * digit
```

```
                n //= 10
```

```
            n = total
```

```
        return True
```

REMOVE LINKED LIST ELEMENTS

class Solution:

def isHappy(self, n):

seen = set()

while n != 1:

if n in seen:

return False

seen.add(n)

total = 0

while n > 0:

digit = n % 10

total += digit * digit

n //= 10

n = total

return True

COUNT PRIMES

```
class Solution(object):  
    def countPrimes(self, n):  
        if n <= 2:  
            return 0  
  
        is_prime = [True] * n  
        is_prime[0] = is_prime[1] = False  
  
        i = 2  
        while i * i < n:  
            if is_prime[i]:  
                for j in range(i * i, n, i):  
                    is_prime[j] = False  
            i += 1  
  
        return sum(is_prime)
```

ISOMORPHIC STRINGS

```
class Solution:
```

```
    def isIsomorphic(self, s, t):
```

```
        s_map = {}
```

```
        t_map = {}
```

```
        for i in range(len(s)):
```

```
            a = s[i]
```

```
            b = t[i]
```

```
            if s_map.get(a, -1) != t_map.get(b, -1):
```

```
                return False
```

```
            s_map[a] = i
```

```
            t_map[b] = i
```

```
        return True
```

REVERSE LINKED LIST

```
class Solution:
```

```
    def reverseList(self, head):
```

```
        prev = None
```

```
        current = head
```

```
        while current:
```

```
            next_node = current.next
```

```
            current.next = prev
```

```
            prev = current
```

```
            current = next_node
```

```
        return prev
```

MINIMUM SIZE SUBARRAY SUM

```
class Solution {  
    public int minSubArrayLen(int target, int[] nums) {  
  
        int left = 0;  
  
        int sum = 0;  
  
        int minLength = Integer.MAX_VALUE;  
  
        for (int right = 0; right < nums.length; right++) {  
            sum += nums[right];  
  
            while (sum >= target) {  
                minLength = Math.min(minLength, right - left + 1);  
                sum -= nums[left];  
                left++;  
            }  
        }  
  
        return (minLength == Integer.MAX_VALUE) ? 0 : minLength;  
    }  
}
```

CONTAINS DUPLICATE - 1

```
class Solution(object):  
    def containsDuplicate(self, nums):  
        """  
        :type nums: List[int]  
        :rtype: bool  
        """  
  
        seen = set()  
  
        for num in nums:  
            if num in seen:  
                return True  
            seen.add(num)  
  
        return False
```

CONTAINS DUPLICATE - 2

class Solution:

```
def containsNearbyDuplicate(self, nums, k):
```

```
    last = {}
```

```
    for i in range(len(nums)):
```

```
        if nums[i] in last:
```

```
            if i - last[nums[i]] <= k:
```

```
                return True
```

```
            last[nums[i]] = i
```

```
    return False
```


Implement Stack using Queues

```
class MyStack:
```

```
    def __init__(self):
```

```
        self.q1 = []
```

```
        self.q2 = []
```

```
    def push(self, x):
```

```
        self.q2.append(x)
```

```
        while self.q1:
```

```
            self.q2.append(self.q1.pop(0))
```

```
        self.q1, self.q2 = self.q2, self.q1
```

```
    def pop(self):
```

```
        return self.q1.pop(0)
```

```
    def top(self):
```

```
        return self.q1[0]
```

```
    def empty(self):
```

```
        return len(self.q1) == 0
```

INVERT BINARY TREE

```
class Solution:
```

```
    def invertTree(self, root):
```

```
        if root is None:
```

```
            return None
```

```
        root.left, root.right = root.right, root.left
```

```
        self.invertTree(root.left)
```

```
        self.invertTree(root.right)
```

```
    return root
```